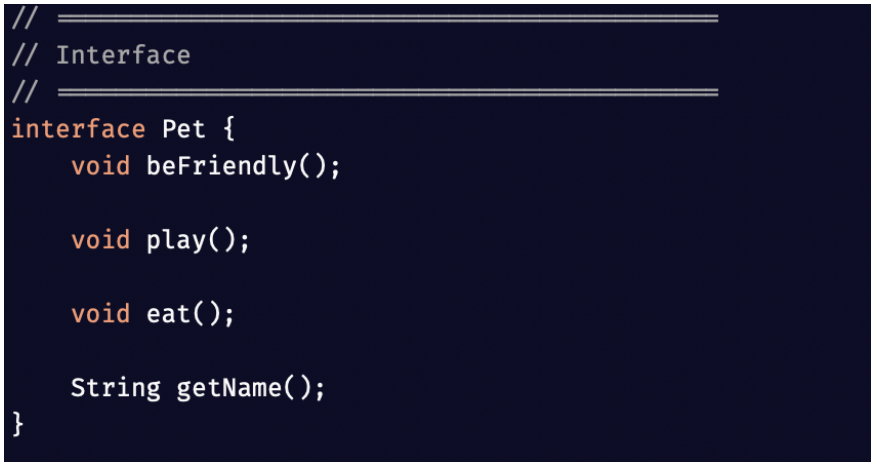
Miles Porter – Tu/Th

CS 2050 Technical Documentation – Module 03

Interfaces
Abstract Classes vs Interfaces
Array Lists
Input Validation
Input from File
Stacks
Queues
Generics
Singly Linked Lists
Doubly Linked Lists

Interfaces

- An interface defines a contract that tells implementing classes which methods they must provide.
- Interfaces are useful when different classes need to share the same behavior without sharing the same parent class.
- An interface focuses on what a class must do, not how it does it.
- Interfaces improve flexibility because multiple classes can implement the same contract in different ways.



```
// ═══════════════════════════════════════════
// Interface
// ═══════════════════════════════════════════
interface Pet {
    void beFriendly();

    void play();

    void eat();

    String getName();
}
```

```
class Bulldog extends Animal implements Pet {
    public Bulldog(String name) {
        super(name);
    }

    @Override
    public void beFriendly() {
        System.out.println(getName() + " wags tail and leans on your leg.");
    }

    @Override
    public void play() {
        System.out.println(getName() + " plays tug-of-war.");
    }

    @Override
    public void eat() {
        System.out.println(getName() + " munches crunchy kibble.");
    }
```

Abstract Classes vs Interfaces

- An abstract class is used when related classes should share common fields or methods.
- An interface is used when classes only need to share required behavior.
- Abstract classes can contain both abstract methods and fully implemented methods.
- Interfaces are better for defining capabilities that many unrelated classes can share.
- A class can only extend one abstract class, but it can implement multiple interfaces.

- I would use an abstract class for a strong parent-child relationship and an interface for shared abilities.

```java
abstract class Vehicle
{

    private String brand;
    private int speed;

    public Vehicle(String brand, int speed)
    {
        this.brand = brand;
        this.speed = speed;
    }

    public String getBrand()
    {
        return brand;
    }

    public int getSpeed()
    {
        return speed;
    }

    public void accelerate(int increase)
    {
        speed += increase;
        System.out.println(brand + " accelerates to " + speed + " mph.");
    }

    public abstract void refuel();

}
```

```java
// ═══════════════════════════════════════════
// Abstract superclass
// ═══════════════════════════════════════════
abstract class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Array Lists

- An ArrayList is a resizable collection that can grow or shrink as needed.
- ArrayList is useful when the number of items is not known ahead of time.
- Elements can be added, removed, searched, and accessed by index.
- ArrayList is easier to manage than a normal array when the collection size changes often.
- add() inserts a new element into the list.
- get() accesses an element by index.
- contains() checks whether a value exists in the list.
- remove() deletes an element from the list.
- size() returns the current number of items in the list.

```
ArrayList<Vehicle> fleet = new ArrayList<>();

fleet.add(new ElectricCar("Hyundai Ioniq5", 60));
fleet.add(new GasCar("Honda Civic", 50));
fleet.add(new HybridCar("Toyota Prius", 55));

System.out.println("=== Fleet Demonstration ===\n");

for (Vehicle car : fleet)
{

    System.out.println("Vehicle: " + car.getBrand());

    car.accelerate(5);
    car.refuel();

    System.out.println();
}
```

Input Validation

- Input validation prevents bad data from crashing the program.
- Validation checks whether the user entered the right type of value before processing it.
- Validation is important because users may enter letters when numbers are expected.
- Re-prompting the user makes the program more reliable and user-friendly.
- hasNextInt() checks whether the next input is actually an integer.
- kb.next() clears the invalid token so the loop can continue.
- This prevents the program from failing on incorrect input.

```
if (scanner.hasNextInt()) {
    int choice = scanner.nextInt();
    scanner.nextLine(); // consume newline
```

```
boolean validInput = false;

while (!validInput) {
    displayMenu("Select an issue type:", issueTypes);
    System.out.print("Enter your choice: ");

    if (scanner.hasNextInt()) {
        choice = scanner.nextInt();
        scanner.nextLine();

        if (choice ⩾ 1 && choice ⩽ issueTypes.length) {
            validInput = true;
        } else {
            System.out.println("Invalid choice! Please enter 1-" + issueTypes.length + ".");
        }
    } else {
        System.out.println("Invalid input! Please enter a number.");
        scanner.next();
    }
}

return issueTypes[choice - 1];
```

Input from File

- File reading allows a program to load saved data instead of requiring manual input every time.
- The File class is used to locate the file on the system.
- The Scanner class can read the file line by line.
- Structured file data can be split into parts and used to build objects.
- hasNextLine() checks whether there is more data to read.
- nextLine() reads one full line from the file.
- split(",") separates structured text into usable pieces.
- This approach is useful when loading objects from CSV-style files.

```
while (scanner.hasNextLine()) {
    String line = scanner.nextLine().trim();
    lineNumber++;

    if (line.isEmpty()) {
        continue;
    }

    String[] data = line.split(",");

    if (data.length != 4) {
        System.out.println("Skipping line " + lineNumber + ": incorrect number of fields.");
        continue;
    }

    String droneType = data[0].trim();
    String manufacturerName = data[1].trim();
    String manufacturedYearString = data[2].trim();
    String payloadKgString = data[3].trim();
```

Stacks

- A stack is a linear data structure that follows LIFO, which means last in, first out.
- The most recently added item is the first one removed.
- Stacks are useful for undo features, browser history, and expression processing.
- Stack operations are usually simple and fast because all work happens at the top.
- push() adds an item to the top of the stack.
- pop() removes and returns the top item.
- peek() returns the top item without removing it.
- In this example, "Page 3" is removed first because it was added last.

```
Stack<String> stackTwo = new Stack<>();
System.out.println("Pushing 3 strings to Stack");
stackTwo.push("String One");
stackTwo.push("String Two");
stackTwo.push("String Three");
System.out.println(stackTwo);

System.out.println("Popping from Stack Two");
stackTwo.pop();
System.out.println(stackTwo);
stackTwo.pop();
System.out.println(stackTwo);

System.out.println("Peeking Stack Two");
System.out.println(stackTwo.peek());
```

```
Pushing 3 strings to Stack
bottom → [String One, String Two, String Three] ← top
Popping from Stack Two
bottom → [String One, String Two] ← top
bottom → [String One] ← top
Peeking Stack Two
String One
```

Queues

- A queue is a linear data structure that follows FIFO, which means first in, first out.
- The earliest added item is the first one removed.
- Queues are useful for task scheduling, waiting lines, and print jobs.
- A queue processes items in the same order they arrived.
- peek() returns the front item without removing it.

```java
class CustomerQueue {

    ArrayList<Customer> queue;

    public CustomerQueue() {
        queue = new ArrayList<>();
    }

    public void displayQueue() {
        for (Customer customer : queue) {
            System.out.println(customer);
        }

    }

    public void enqueue(Customer customer) {
        queue.add(customer);

    }

    public Customer dequeue() {
        int topIndex = queue.size() - 1;
        return queue.remove(topIndex);
    }

    public boolean isEmpty() {
        return queue.isEmpty();
    }

    public Customer peek() {
        int topIndex = queue.size() - 1;
        return queue.get(topIndex);
    }
```

Generics

- Generics allow classes and methods to work with different data types safely.
- Generics improve reusability because the same code can work with many object types.
- Generics improve type safety because the compiler checks data types at compile time.
- Generics reduce the need for casting.

Singly Linked Lists

- A singly linked list stores data in nodes connected by references.
- Each node stores a value and a reference to the next node.
- Singly linked lists are good for insert and delete operations when shifting elements would be inefficient.
- Traversal moves through the list one node at a time from front to back.
- head stores the starting point of the list.
- Each node points only to the next node in the chain.
- traverse() visits each node until reaching null.

```
public void insertNode(int number)
{
    NodeFix newNode = new NodeFix(number);
    NodeFix current = head;
    NodeFix previous = null;

    while (current ≠ null && current.data < number)
    {
        previous = current;
        current = current.next;
    }

    if (previous == null)
    {
        newNode.next = head;
        head = newNode;
    } else
    {
        // The original code only did previous.next = newNode,
        // which inserted the new node but disconnected the rest of the list.
        // You must point the new node to current first, then link previous to newNode.
        newNode.next = current;
        previous.next = newNode;
    }
```

```
public void deleteNode(int number)
{
    NodeFix current = head;
    NodeFix previous = null;

    // The original loop used current.next ≠ null, which could crash on an empty list
    // and also failed to properly check the last node.
    // Using current ≠ null safely checks every node in the list.
    while (current ≠ null && current.data ≠ number)
    {
        previous = current;
        current = current.next;
    }
    // If current is null, the value was not found.
    // The original code would continue and could throw a NullPointerException.
    if (current == null) {
        return;
    }
    if (previous == null)
    {
        head = current.next;
    } else
    {
        previous.next = current.next; // Bug #5: Should be previous.next = current.next
    }
}
```

```java
public void printList()
{
    NodeFix current = head;
    while (current ≠ null)
    {
        System.out.print(current.data + " → ");
        current = current.next;
    }
    System.out.println("null");
}
```

Doubly Linked Lists

- A doubly linked list gives each node both a next reference and a prev reference.
- Doubly linked lists allow movement in both forward and backward directions.
- Deletion is more flexible because each node can access its neighboring nodes directly.
- Doubly linked lists use more memory because every node stores two references.
- next points to the next node in the list.
- prev points to the previous node in the list.
- This structure improves navigation but costs more memory than a singly linked list.